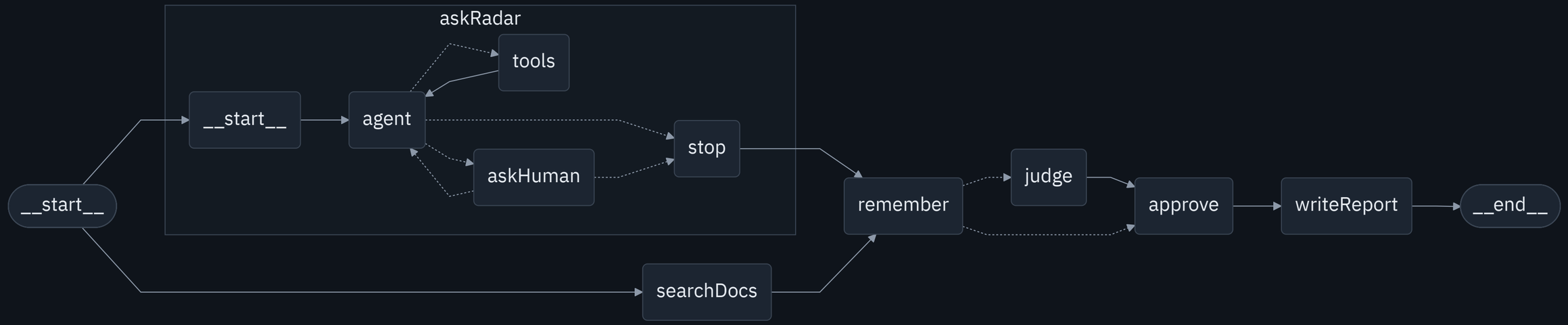
보고 한 줄이 들어오면, 수집하고, 기억하고, 판정하고, 사람이 결정한다



- 규정집 Worker 와 레이더 부하 Agent 가 **동시에** 자료를 모은다
- 전에 같은 보고를 판단했으면 기억이 답한다 (Jev 생략)
- 없으면 Jev 가 출격 / 감시 / 무시 의견을 낸다
- 마지막 버튼은 언제나 사람이 누른다
- 오늘 이 그래프를 **거의 빈 Agent 에서 8단계로** 같이 붙여 완성한다
- 점선은 조건에 따라 갈리는 선, 실선은 항상 가는 선

소개

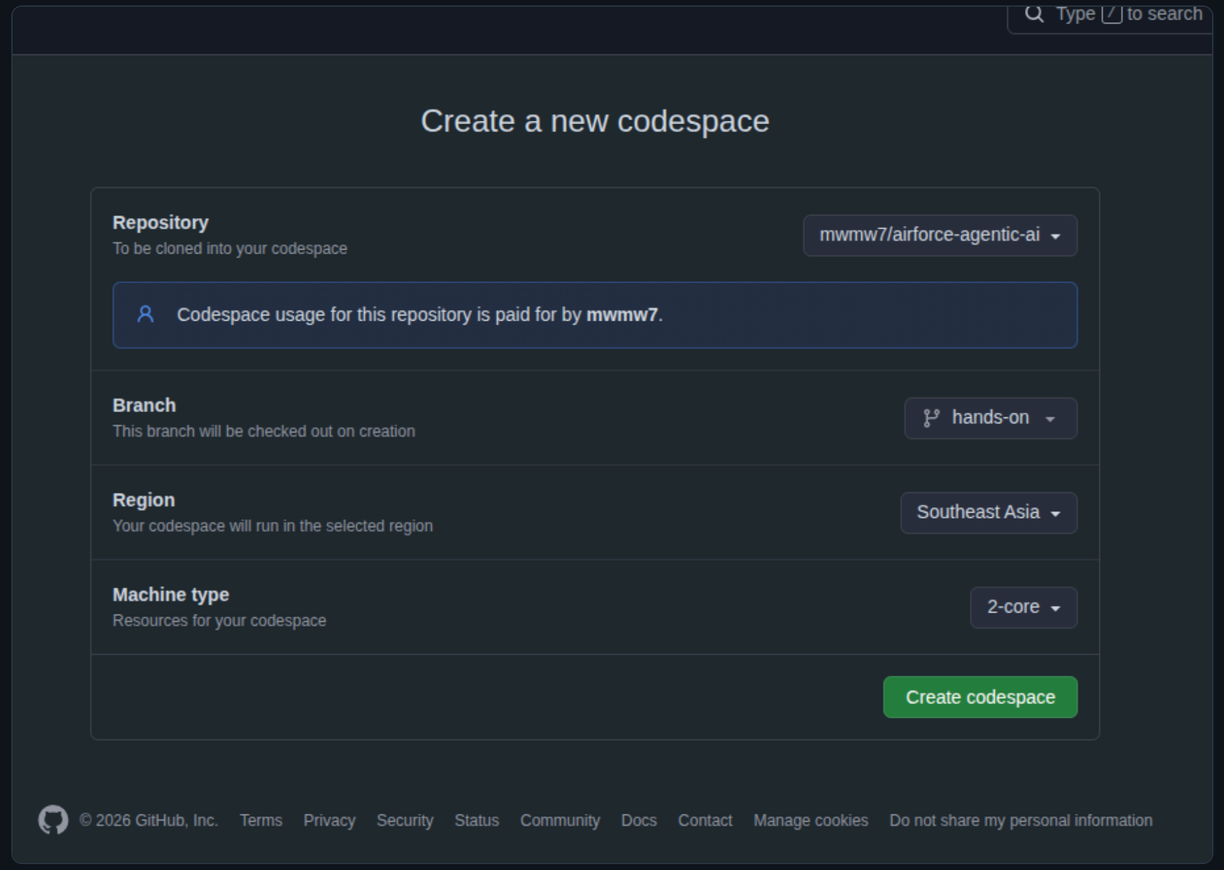
최호섭

삼성전자 R&D센터 → 툰스퀘어 CTO

Tooning Edu	웹툰 제작 툴
Tooning Plus	Tooning 의 유료 고급 기능
Deev	캐릭터 챗 서비스. 오늘 오후에 이 서비스의 실제 지시문 탐지 코드를 봅니다

branden@tooning.io

시작은 링크 하나: Codespaces 에서 hands-on 브랜치 열기



링크를 열면 보이는 화면 — 저장소 · 브랜치 hands-on · 2-core 가 미리 골라져 있다. 초록 단추만 누른다

> `$ https://codespaces.new/mwmw7/airforce-agentic-ai?ref=hands-on` GitHub 로그인만 있으면 된다

- 브랜치가 **hands-on** 인지 확인하고 **Create codespace**
- 브라우저에 VS Code 가 뜨면 설치 · .env · 환경 점검이 **자동으로** 끝난다 (`.devcontainer/`). "준비 끝" 이 보이면 `pnpm agent`
- 개인 계정은 달마다 2-core 60시간이 무료. 이틀 교육은 10시간이면 끝
- 30분 안 쓰면 자동으로 멈춘다. 다시 열면 그 자리부터 (점심 뒤)
- 노트북에 직접 하고 싶으면 다음 장의 `git clone`

fork 는 안 해도 된다. 끝나고 남기고 싶으면 push 할 때 자동으로 내 fork 가 생긴다

오늘의 규칙: 같이 하나씩 붙이고, 매번 다시 돌린다

```
$ git clone -b hands-on https://github.com/mwmw7/airforce-agentic-ai 노트북이면.
Codespaces 는 앞 장
```

```
$ pnpm install && pnpm setup:check .env 는 알아서 만들어 준다
```

```
$ pnpm agent 빈칸이 비어 있어도 끝까지 돈다
```

- 문제는 **TODO 1 ~ 8**. 단계 번호 = TODO 번호. 파일 안에서 `// ===== TODO n` 을 찾는다
- 강사가 화면에서 치면 같이 친다. 채운 뒤 `pnpm agent` → 화면에 새 줄이 생긴다
- 키가 하나도 없어도 mock 모드로 전부 돈다

늦었으면	상태
`pnpm step 1`	1 ~ 8 전부 남음 (처음 상태)
`pnpm step 4`	1 ~ 3 풀림, 4번을 풀 차례
`pnpm step 8`	1 ~ 7 풀림, 8번만 남음
`pnpm step done`	전부 풀림 (main 과 같음)

강사가 "4단계 시작합니다" 하면 늦은 사람도 같은 숫자 4. 정답은 main 브랜치에서 가져오고, 빈칸이 든 파일만 덮어쓴다 (먼저 묻는다)

오늘 일정

시간	단계	같이 붙이는 것	공군 요청 항목
09:00 ~ 09:50	0 · 1 · 2	첫 실행, 레이더 부하 데려오기, 부하에게 도구 하나 더	Subagent · Agent Loop · Harness · Tool
10:05 ~ 11:00	3 · 4 · 5	규정집 찾기, 날씨 MCP, 동시에 모으기	RAG · MCP · Task Decomposition
11:15 ~ 12:00	6 · 7 · 8	Jev 판정, 사람 답으로 이어가기, 기억	판정 모델 · HITL · Checkpoint · Memory
13:00 ~ 14:10	시연	자료 속 지시문, 모델 교체, 실행 기록	Guardrail · Sandbox · Trace
14:30 ~ 16:00	특강 (이현우)	Langfuse 로 검증과 활용	Evaluation · Observability

휴식 09:50 · 11:00 · 14:10 — 휴식 뒤 늦은 사람은 `pnpm step 3` · `pnpm step 6`

0단계 · 첫 실행: 근거가 텅 빈 담당자 화면

```
TOOL    search_docs(보고 그대로)
HUMAN   종합 — 담당자가 결정합니다
          보고
          관련 규정
            · 관련 규정 없음
          기억
            · 비슷한 보고 없음
          Jev 판정
            · watch · 확신 0% · 위험도 0단계 (전혀 안 위험함, 심심한 하루) …
          코드 권고
            · Jev 도 헷갈려함 → 사람이 한 번 더 확인
          모델이 정리한 사실
            · (TODO 1: 레이더 부하가 아직 없어서 확인한 사실이 없습니다)
STOP    답(w)을 받았지만 그래프를 이어가지 않았습니다 — TODO 7
DECIDE  Jev 도 헷갈려함 → 사람이 한 번 더 확인  ← Jev watch 0% + 코드 규칙
```

```
$ pnpm agent 3
```

- 보고는 들어왔는데 **눈(레이더)이 없다**. 규정도, 날씨도, 기억도 없다
- Jev 자리는 "확신 0%" — 코드 규칙이 "사람이 한 번 더 확인" 으로 보낸다
- 마지막 줄: 사람 답을 받고도 이어가지 않는다 (TODO 7)
- 오늘 8단계 동안 이 화면에 줄이 하나씩 늘어난다

이게 보이면 성공

끝까지 돌고 `[DECIDE]` 가 찍힌다. 빈칸이 비어도 프로그램은 멈추지 않는다

1단계 · 레이더 담당 부하 Agent 데려오기

```
- .addNode("askRadar", async () => ({ facts: "(TODO 1: 레이더 부하가 아직 없…
+ .addNode("askRadar", radarAgent)                    // Subagent: 안에 age…
```

```
$ pnpm agent 1
```

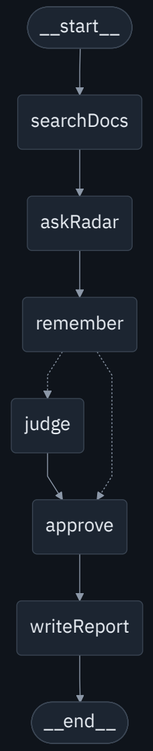
- `ask-radar.ts` 는 1일차에 만든 것과 같은 **도구 쓰는 Agent** 하나다
- 함수 자리에 `compile()` 된 그래프를 넣으면 그게 부하(Subagent)다
- 부모와 이름이 겹치는 칸(facts · evidence · steps)만 돌아온다. 부하의 대화는 부하 안에

```
TOOL  search_docs(보고 그대로)
SUB   ┌─ askRadar 레이더 부하 Agent (Subagent) — 여기부터 ┃ 줄은 부하의 …
┃ LLM  1회째 → 도구 쓰기: look_radar  35ms
┃ TOOL  look_radar(어디: 서울 상공에 정체불명의 둥근 비행체가 떠 있어요. …
┃ LLM  2회째 → 도구 쓰기: look_closer  1ms
┃ TOOL  look_closer(어디: 서울 상공에 정체불명의 둥근 비행체가 떠 있어요.…
┃ LLM  3회째 → 도구 쓰기: look_weather  1ms
┃ LLM  4회째 → 사실 정리 (도구 안 씀)  0ms
SUB   └─ askRadar 끝 · 모델 4회 호출, 사실 정리 완료 → 부모에게
```

이게 보이면 성공

`[SUB] ┌─ askRadar` 띠가 열리고, 담당자 화면에 **레이더 · 확대** 줄이 생긴다

1단계 전후 — 그래프에서 무엇이 바뀌었나?



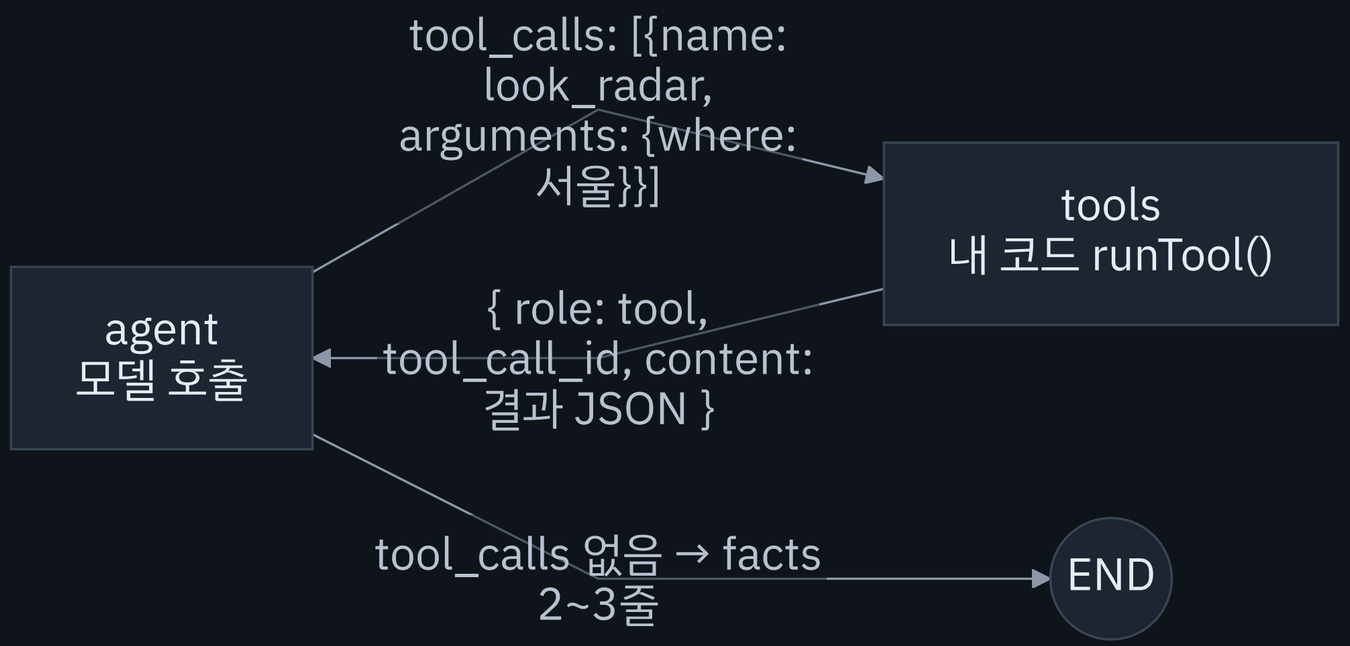
채우기 전 (`pnpm step 1`): askRadar 는 함수 하나짜리 가짜 부하



채운 뒤 (`pnpm step 2`): askRadar 가 agent ⇄ tools 루프로 펼쳐진다

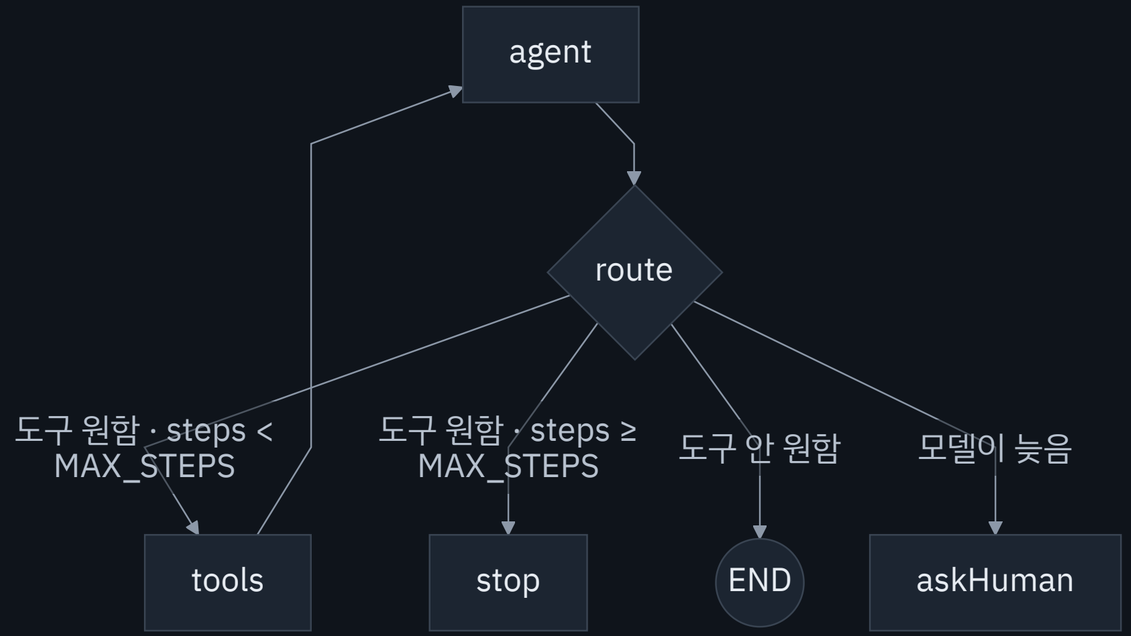
- 바뀐 건 노드 **하나의 속**. 바깥 선(searchDocs → askRadar → remember)은 그대로다
- 오늘 그래프 모양이 바뀌는 건 두 번뿐: 지금(1단계, 노드)과 5단계(선). 나머지 단계는 노드 안의 코드만 채운다
- 그림 다시 뽑기: `pnpm see:steps` → `docs/graphs/step-*.md`

부하 안을 열면? — 1일차 그 루프



- 모델은 **JSON 으로 "이 도구를 써 달라"** 고 말한다
- 실행은 내 코드 `runTool` 이 한다. 모델은 파일도 네트워크도 못 만진다
- 결과는 `role: "tool"` 메시지로 다시 모델에게
- 이 부분은 완성 코드로 준다 — 어제(1일차) 직접 만든 것과 같다

누가 멈추나? — 모델이 아니라 코드




```
if (!wantsTool) return END;                  // 사실 정리 끝 → 부모에게
if (state.steps >= MAX_STEPS) return "stop"; // 내 코드가 끊는다
return "tools";
```

```
SUB     ┌ askRadar 레이더 부하 Agent (Subagent) — 여기부터 ┃ 줄은 부하의 …
┃ LLM   1회째 → 도구 쓰기: look_radar  36ms
┃ STOP  모델을 이미 1번 불렀습니다. 더 부르지 않고 내 코드가 멈췄습니다. …
SUB     └ askRadar 끝 · 강제 중단
MEMORY  [file] 같은 사례 없음 (가장 비슷한 것 일치도 0.03 < 0.9) → Jev 에게…
MEMORY  [file] recall · Jev 참고용 비슷한 기억 (겹치는 단어 수) → 1건
JEV     [local] System One 판정 · local 규칙 판정기 · 질문 3개 한 번에 · 1m…
HUMAN   종합 — 담당자가 결정합니다
MEMORY  [file] retain → .hindsight/control-room.json 에 1건 (실행번호 loop-…
```

```
$ pnpm agent:limit MAX_STEPS=1
```

이게 보이면 성공

`[STOP]` 은 부하만 끝낸다. 부모는 규정집 결과만 들고 판정과 담당자 화면까지 간다

칠판(State)은 어떻게 쌓이나?

```
NODE ▶ askRadar › agent 시작 · ← branch:to:agent
     │ steps 0
NODE ◀ askRadar › agent 끝 · 27ms
     │ steps 1
NODE ▶ askRadar › tools 시작 · ← branch:to:tools
     │ steps 1
NODE ▶ askRadar › agent 시작 · ← branch:to:agent
     │ steps 1
NODE ◀ askRadar › agent 끝 · 1ms
     │ steps 1
NODE ▶ askRadar › tools 시작 · ← branch:to:tools
     │ steps 2
```

```
$ pnpm agent 3 --watch
```

- agent 노드는 매번 `steps: 1` 만 돌려준다
- 칠판에는 1, 2, 3 으로 쌓인다. **reducer** 가 더하기 때문
- 칸마다 규칙이 다르다: 더하기(steps) · 이어 붙이기(messages, evidence) · 덮어쓰기(facts)
- 노드는 "내가 바꾼 칸" 만 돌려주고, 합치는 건 LangGraph 가 한다

```
steps:    Annotation<number>({ reducer: (prev, next) => prev + next })
messages: Annotation<Msg[]>({ reducer: (prev, next) => [...prev, ...next] })
facts:    Annotation<string>()   // 덮어쓰기
```

모델이 늦으면?

```
LLM     실패 radar.step1 0ms
HUMAN   가짜 모델 이 60초 안에 답하지 않았습니다 (1회째 호출)
SUB     └ askRadar 끝 · 담당자가 skip → 사실 없이 부모에게
HUMAN   종합 — 담당자가 결정합니다
DECIDE  감시 강화 — 사람이 결정  ← Jev scramble 90% + 코드 규칙
```

```
$ LLM_MOCK_SLOW=1 pnpm agent 4
```

- 한 번 호출이 `LLM_TIMEOUT_SEC` 를 넘으면 끊는다
- 끊은 뒤 뭘 할지는 코드가 정하지 않고 **사람에게 묻는다**: retry / skip / stop
- skip 이면 사실 없이 판정으로, stop 이면 전부 중단
- 실행 전체가 `RUN_TIMEOUT_SEC` 를 넘어도 끊는다

2단계 · 부하에게 도구 하나 더 — 비행 계획 조회

```
- // { type: "function", function: { name: "look_flight_plan", description: "…", paramete
+ { type: "function", function: { name: "look_flight_plan", description: "오늘 미리 통보되…
+ parameters: { type: "object", properties: { where: { type: "string", description: "장소
```

```
- // if (name === "look_flight_plan") { … }
+ if (name === "look_flight_plan") {
+ const where = String(args.where ?? "");
+ const plans: RadarRow[] = readJson("flight-plans.json");
+ const found = plans.filter((p) => inRegion(p, where));
+ return JSON.stringify({ 비행계획: found.length ? found.map(brief) : [`'${where.slice(0, …
+ }
```

```
$ pnpm agent 3 잠실 드론
```

```
LLM  3회째 → 도구 쓰기: look_flight_plan  0ms
TOOL look_flight_plan(어디: 잠실 상공에 밤중에 드론 수십 대가 떼로 날…
       |    · 어디 잠실 상공  |  뭐가 드론 쇼  |  언제 -  |  허가 신청 없…
     | 비행 계획
     |   · 잠실 상공 · 드론 쇼 · - · 신청 없음 (오늘 접수된 드론 쇼 없음…
```

- 도구 = **설명(모델이 읽는 것) + 실행(내 코드)**. 둘 다 내가 쓴다
- 모델은 description 을 읽고 "언제 쓸지" 정한다. 설명이 선택을 정한다
- 자료 `data/api/flight-plans.json` : 잠실 드론 쇼는 신청 없음

이게 보이면 성공

`[TOOL] look_flight_plan` 이 찍히고 담당자 화면에 **비행 계획** 줄이 생긴다

모델이 모르는 문서는?



- RAG = 모델이 답하기 전에 **자료를 찾아서 넣어 주는** 구조
- 어떻게 찾느냐는 자유. 오늘은 가장 단순한 겹치는 단어 수
- 임베딩으로 바꾸고 싶으면 점수 함수 하나만 바꾼다. 그래프는 그대로
- 규정집은 `data/docs/rulebook.md` 열 줄, 전부 가상

3단계 · 규정 한 줄에 점수 매기기

```
- const scored = rules.map((rule) => ({ rule, score: 0 }));
+ const scored = rules.map((rule) => ({
+ rule,
+ score: words(rule).filter((word) => question.includes(word)).length,
+ }));
```

```
$ pnpm agent 2
```

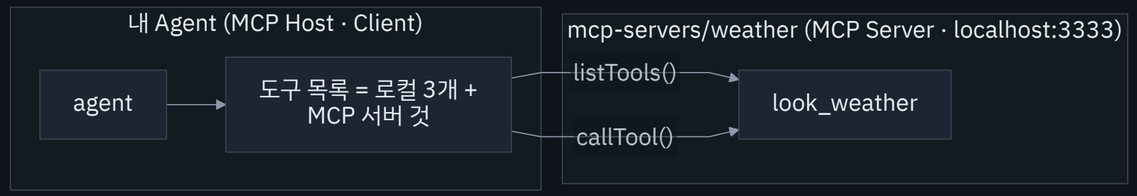
```
TOOL search_docs(보고 그대로)
  | 3. 풍선, 연, 새 떼는 위협이 아니다. 공항 쪽으로 가면 공항에 알리…
  | 2. 무전 응답이 없어도 느리게 바람 따라 움직이는 물체는 풍선이나 …
```

```
  | 관련 규정
  |   · 3. 풍선, 연, 새 떼는 위협이 아니다. 공항 쪽으로 가면 공항에 …
  |   · 2. 무전 응답이 없어도 느리게 바람 따라 움직이는 물체는 풍선이…
```

이게 보이면 성공

담당자 화면 **관련 규정** 에 풍선 규정이 올라온다. 점수가 전부 0 이면 "관련 규정 없음"

도구를 다른 프로세스에 두면?



- MCP = 도구를 **다른 프로세스**에 두고 표준 규약으로 부르는 것
- 서버: `registerTool` 로 이름 · 설명 · 입력 모양 · 실행 함수를 등록 (4-1)
- 클라이언트: 접속 → 목록 받기 → 실행 (4-2)
- 모델 입장에선 2단계에서 만든 로컬 도구와 구분이 없다. 같은 목록에 섞여 있을 뿐

4단계 · 날씨를 MCP 로 — 서버에 등록, 클라이언트에서 목록 받고 실행

```
- // server.registerTool("look_weather", { description: "...", inputSchema: { where: z.st
+ server.registerTool(
+ "look_weather",
+ {
+ description: "그 장소의 지금 날씨(하늘·구름·바람·시야·비행 가능 여부)를 본다. 보고에 나…
+ inputSchema: { where: z.string().describe("장소 또는 보고 문장 그대로. 예: 한강 / 잠실 …
+ },
+ async ({ where }) => ({ content: [{ type: "text", text: await lookWeather(where) }] }),
+ );
```

```
$ pnpm mcp:weather 다른 터미널에서 먼저
```

```
$ pnpm mcp:inspect http://localhost:3333/mcp
```

```
- return [];
+ const { tools } = await c.listTools();
+ mcpToolNames = new Set(tools.map((t) => t.name));
+ log("mcp", `날씨 서버 연결 OK — 도구 ${tools.length}개 받음: ${tools.map((t) => t.name).…
+ return tools.map((t) => ({
+ type: "function",
+ function: { name: t.name, description: t.description ?? "", parameters: t.inputSchema a
+ }));
```

```
- return JSON.stringify({ error: "TODO 4-2 미구현" });
+ try {
+ const result = await c.callTool({ name, arguments: args }, undefined, { timeout: 120_00
+ const content = result.content as { type: string; text?: string }[];
+ return content.filter((x) => x.type === "text").map((x) => x.text ?? "").join("\n");
+ } catch (e) {
+ // 서버가 늦거나 죽어도 agent 는 멈추지 않는다. 결과 대신 사정을 돌려주면 모델이 "날씨 …
+ return JSON.stringify({ error: `MCP 도구 ${name} 실패: ${(e as Error).message}` });
+ }
```

```
$ pnpm agent
```

```
MCP 날씨 서버 연결 OK — 도구 1개 받음: look_weather
MCP look_weather(어디: 서울 상공에 정체불명의 둥근 비행체가 떠 있어요…
```

이게 보이면 성공

`[MCP] 날씨 서버 연결 OK — 도구 1개`, 담당자 화면에 **날씨** 줄

서버를 끄면?

```
MCP     날씨 서버 연결 실패 (http://localhost:3999/mcp) — 다른 터미널에서…
LLM     1회째 → 도구 쓰기: look_radar  18ms
LLM     2회째 → 도구 쓰기: look_closer  0ms
LLM     3회째 → 사실 정리 (도구 안 씀)  0ms
SUB     └ askRadar 끝 · 모델 3회 호출, 사실 정리 완료 → 부모에게
DECIDE  감시 강화 — 사람이 결정  ← 기억으로 판단 · Jev 생략
```

```
$ pnpm agent 2 날씨 서버를 끈 채로
```

- 연결 실패는 **에러가 아니라 안내 한 줄**이다
- 도구 목록에서 날씨만 빠지고 Agent 는 계속 간다
- 외부 의존이 죽어도 Agent 가 사는 것은 코드로 정해 둔 일이다
- 어디서 그렇게 했나: `labs/agent/mcp.ts` 의 `connectMcp` 가 null 을 돌려준다

LangGraph 사용자가 가장 많이 쓰는 그래프 다섯 가지

패턴	모양	한 줄 비유	우리 코드
Tool-calling 루프 (ReAct)	agent ⇄ tools 왕복	혼자 일하는 직원	있음 — ask-radar.ts (1단계)
Supervisor	감독 모델이 부하 여럿 중 누구에게 넘길지 고름	팀장이 팀원을 지명	없음 — 다음 단계
Orchestrator–Workers	일을 쪼개 동시에 뿌리고 합침	팀장이 일을 쪼개 동시에 배분	있음 — main.ts (5단계)
Router	입력 보고 갈림길 하나, 루프 없음	안내 데스크	route 함수가 축소판
Evaluator–Optimizer	만들기 → 평가 → 다시 만들기	퇴고하는 작가	실제 사례 시연: 3D 이펙트 생성 → 렌더 → 평가 → 재생성

interrupt 로 사람이 끼어드는 것은 패턴이 아니라 어느 패턴에나 끼우는 공통 부품이다. LangChain 이 권하는 순서: ReAct 하나로 시작하고, 도구가 열 개를 넘거나 역할이 갈리면 쪼개라

Worker 와 Subagent 는 뭐가 다른가?

	Worker	Subagent
하는 일	정해진 일 하나	목표 하나를 받아 알아서 끝냄
모델 호출	없음	있음, 반복 (도구를 고르고 또 고름)
실행 횟수	항상 같음	상황마다 다름
코드 모양	함수 하나	그래프 하나 (agent ⇄ tools)
우리 코드	searchDocs (3단계)	askRadar (1단계)

- 복사기와 인턴. 복사기는 몇 장 나올지 미리 알고, 인턴은 몇 번 왔다 갔다 할지 자기가 정한다
- **LangGraph 에겐 둘 다 노드 하나다.** 안에 모델 루프가 있으면 우리가 부하라고 부를 뿐
- 선 잇기 · 체크포인트 · interrupt 코드는 둘이 완전히 같다

```
.addNode("searchDocs", searchDocsNode)   // 함수
.addNode("askRadar",   radarAgent)       // compile() 된 그래프
```

5단계 · 순차를 동시로

```
- .addEdge(START, "searchDocs")
- .addEdge("searchDocs", "askRadar")
- .addEdge("askRadar", "remember")
+ .addEdge(START, "searchDocs")
+ .addEdge(START, "askRadar")
+ .addEdge(["searchDocs", "askRadar"], "remember")
```

```
$ pnpm agent 1 --watch
```

LangGraph 는 한 단계의 노드가 전부 끝나야 다음으로 간다. 배열로 묶으면 "둘 다 끝나야" 가 코드에 보이고, 가지 길이가 달라도 안전하다

```
NODE  ▶ searchDocs 시작 · ← branch:to:searchDocs
NODE  ◀ searchDocs 끝 · 1ms
NODE  ▶ askRadar 시작 · ← branch:to:askRadar
```

채우기 전: 규정집이 끝나야 레이더가 출발

```
NODE  ▶ askRadar 시작 · ← branch:to:askRadar
NODE  ▶ searchDocs 시작 · ← branch:to:searchDocs
NODE  ◀ searchDocs 끝 · 1ms
NODE  ◀ askRadar › agent 끝 · 1ms
NODE  ◀ askRadar 끝 · 2239ms
```

채운 뒤: 둘이 같이 출발, remember 는 둘 다 끝나야

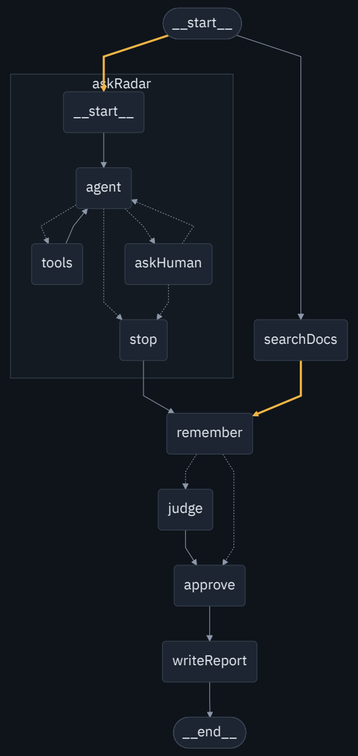
이게 보이면 성공

`▶ askRadar` 와 `▶ searchDocs` 가 나란히 시작하고, remember 가 `← join:searchDocs+askRadar` 로 깨어난다

5단계 전후 — 그래프에서 무엇이 바뀌었나?



채우기 전 (`pnpm step 5`): 규정집 → 레이더 → remember, 한 줄로



채운 뒤 (`pnpm step 6`): START 에서 둘로 갈라졌다가 remember 에서 합류

- 노드는 하나도 안 늘었다. 코드 세 줄을 바꿨고, 그림에선 **searchDocs → askRadar 선이 빠지고 앰버 선 두 개가 생겼다**
- 1단계는 노드 속을, 5단계는 노드 사이의 선을 바꿨다. 오늘 그래프 모양이 바뀌는 건 이 두 번이 전부다

왜 나누나?

이점	하나의 루프였을 때	부하로 나누면	어디서 보이나
작업 기록이 부모에 안 남는다	레이더 조회 대화 20줄이 판정 · 저장 · 화면까지 따라감	20줄은 부하 칠판에만. 부모엔 facts 3줄 + evidence	`--watch` : remember 입력에 messages 가 없다
부하가 멈춰도 부모는 산다	MAX_STEPS 면 전체 종료	부하만 끝나고 규정집 결과로 판정까지	`pnpm agent:limit`
부하 단위로 갈아 끼운다	루프 · 판정 · 승인이 한 파일에	모델 · 키 · 도구를 부하만 바꾸거나 옆에 하나 더	다음 단계
속도	—	**지금 구성에선 없다.** 모델 쓰는 가지가 하나뿐이고, 노트북 GPU 는 요청을 한 줄로 받는다	가지 2개 이상 + Bedrock 일 때

이점이 아닌 것: 판정 정확도는 안 바뀐다(judge 는 facts 만 읽는다) · 코드가 짧아지지 않는다(파일 둘, 칠판 둘) · LangGraph 에겐 그냥 노드 하나

LLM 말고 Jev 를 쓰면?

	LLM	Jev (TypeSafe System One)
돌려주는 것	문장	확률 + 확신도
질문 모양	자유 프롬프트	choice · score · noul 세 종류
속도	초 단위	밀리초 단위
우리 코드	부하의 사실 정리	judge 노드 3질문 (QUESTIONS)

```
const QUESTIONS = {
  action:  { type: "choice", criteria: { scramble, watch, ignore } },
  danger:  { type: "score",  criteria: ["안 위험", "조금", "꽤", "비상"] },
  unknown: { type: "noul",   instructions: "정체가 무엇인지 전혀 짐작이 안 …
};
```

- Jev 는 **의견**만 준다. 출격 / 감시 / 무시를 정하는 규칙은 코드에 있다
- 키가 없으면 `JEV_MODE=local` 로 Jev 자리를 정규식 규칙이 대신한다. 흐름은 똑같이 돈다

6단계 · Jev 에게 묻기

```
- const res: JevResult = { answers: { action: { type: "choice", choice: "watch",
+ const res = await askJev({ report: state.report, facts: state.facts, past }, QU
```

```
$ pnpm agent 1
```

- 질문 세 개는 위에 `QUESTIONS` 로 미리 적혀 있다. 빈칸은 묻는 한 줄
- 비워 두면 "확신 0% 감시" 뿐이라 코드 규칙이 언제나 사람에게 넘긴다

```
JEV   [typesafe] System One 판정 · typesafe · jev-latest · 질문 3개 한 번에 · 215ms
      │ 보낸 근거 state report(66자) · facts(123자) · past(0건)
      │ ① action   choice report 와 facts 를 보고 관제실이 지금 뭘 해야 할까?
      │      ▶ watch      ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  63%  확신 45%
      │        scramble   ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  35%
      │        ignore     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓   2%
      │ ② danger   score  얼마나 위험해 보이나?
      │      → 2.23 / 3   ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  꽤 수상함
      │ ③ unknown  noul   정체가 무엇인지 전혀 짐작이 안 가서 정체부터 알아내야 한다
      │      → 예 (예일 확률) ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  79%
RULE  Jev 는 의견, 결정은 코드 규칙 · 조건 watch + 정체불명 > 50% → 정체부터 알아보기 (레이더…
```

이게 보이면 성공

`[JEV]` 막대 세 개와 `[RULE]` 줄이 찍히고, 담당자 화면 **Jev 판정** 에 확신도 · 위험도 · 정체불명

판정 화면 읽기 · 질문 한 줄이 결과를 바꾼다

- ① action: 셋 중 하나를 고르고 확률을 나란히 준다. 확신(confidence)은 1등과 2등의 차이
- ② danger: 0 ~ 3 점수. 기준 문장 4개를 우리가 적었다
- ③ unknown: 예 / 아니오 확률
- `[RULE]` 줄이 코드 규칙이다: 확신 20% 미만이면 사람에게, scramble 이면 출격 준비, watch 인데 정체불명 50% 이상이면 정체부터

```
- instructions: "정체가 무엇인지 전혀 짐작이 안 가서 정체부터 알아내야 한다"…
+ instructions: "더 알아봐야 해?",
```

```
$ pnpm agent 1  UFO
```

```
$ pnpm agent 4  훈련기
```

해 보기: noul 문장을 모호하게 바꾸면 어느 보고든 70 ~ 80% 가 나온다. 좁혀 쓰면 UFO 75% 근처, 훈련기 10% 이하로 갈린다 (실제 Jev 키가 있어야 차이가 난다)

JEV_MODE 를 바꾸면?

(index)	보고	판정	의견	확신	위험도	결정
0	'1. 서울 상공에 정체불명의 둥근 비행체가 떠…'	'local'	'scramble'	'42%'	'3단계'	'감시 강화'
1	'2. 한강 위로 거대한 오리 모양 풍선이 떠내…'	'local'	'scramble'	'90%'	'3단계'	'감시 강화'
2	'3. 잠실 상공에 밤중에 드론 수십 대가 떼로…'	'local'	'scramble'	'42%'	'3단계'	'감시 강화'
3	'4. 옆 부대 훈련기예요. 아침에 미리 연락 …'	'local'	'scramble'	'42%'	'3단계'	'감시 강화'

JEV_MODE=local — Jev 자리를 정규식 규칙이 대신. 정체불명 보고는 전부 scramble 로 기운다

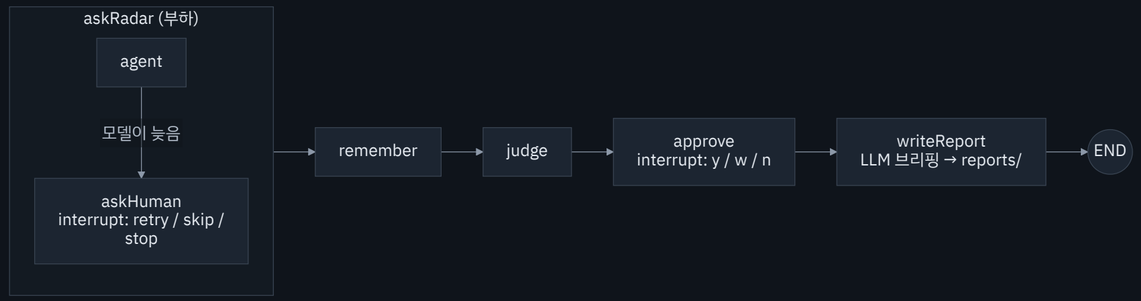
(index)	보고	판정	의견	확신	위험도	결정
0	'1. 서울 상공에 정체불명의 둥근 비행체가 떠…'	'typesafe'	'watch'	'45%'	'2단계'	'감시 강화'
1	'2. 한강 위로 거대한 오리 모양 풍선이 떠내…'	'typesafe'	'watch'	'80%'	'1단계'	'감시 강화'
2	'3. 잠실 상공에 밤중에 드론 수십 대가 떼로…'	'typesafe'	'watch'	'95%'	'2단계'	'감시 강화'
3	'4. 옆 부대 훈련기예요. 아침에 미리 연락 …'	'typesafe'	'ignore'	'97%'	'0단계'	'감시 강화'

JEV_MODE=typesafe — 실제 Jev. 보고마다 의견과 확신이 다르다. 훈련기는 ignore 97%

```
$ JEV_MODE=local pnpm agent --all --answer w
```

```
$ JEV_MODE=typesafe pnpm agent --all --answer w
```

사람이 끼어들면 어디서 멈추나?



- `interrupt(질문)` 한 줄이면 그래프가 거기서 멈추고 질문이 바깥으로 나온다
- 두 곳: 부하 안(모델이 늦을 때), 부모의 approve(언제나)
- approve 는 레이더 · 확대 · 비행 계획 · 날씨 · 규정 · 기억 · Jev 를 한 화면에 모아 보여 주고 묻는다
- Agent 는 권고까지. 버튼은 사람이 누른다

7단계 · 사람의 답으로 멈춘 곳부터 이어가기

```
- const answer = await askPerson(question);
- log("stop", `답(${answer})을 받았지만 그래프를 이어가지 않았습니다 — TODO …
- break; // ← 이대로면 답을 받고도 이어가지 않습니다
+ await run(new Command({ resume: await askPerson(question) }));
```

```
$ pnpm agent
```

- 같은 `thread_id` 로 다시 돌리면 LangGraph 가 멈춘 자리를 찾는다
- 답은 `interrupt()` 의 반환값으로 들어간다. 노드 함수는 처음부터 다시 돌지만 그 줄에서 답을 받는다

```
HUMAN   답: w (--answer 로 미리 정함)
STOP    답(w)을 받았지만 그래프를 이어가지 않았습니다 — TODO 7
DECIDE  Jev 도 헷갈려함 → 사람이 한 번 더 확인  ← Jev watch 0% + 코드 규칙
```

채우기 전: 답을 받고도 끝난다

```
HUMAN   답: w (--answer 로 미리 정함)
DECIDE  감시 강화 — 사람이 결정  ← Jev watch 25% + 코드 규칙
```

채운 뒤: 같은 줄에서 이어져 사람의 결정으로

이게 보이면 성공

`[DECIDE] … — 사람이 결정`

껐다 켜면?

```
SETUP  모델 mock · Jev local · 기억 file
       | 모델 한 번 60초 · 실행 전체 180초 · 최대 8회 · 실행번호 loop-1-17…
HUMAN  종합 — 담당자가 결정합니다
^C  (담당자 확인 중 Ctrl+C)
```

멈춘 실행번호가 SETUP 줄에 있다

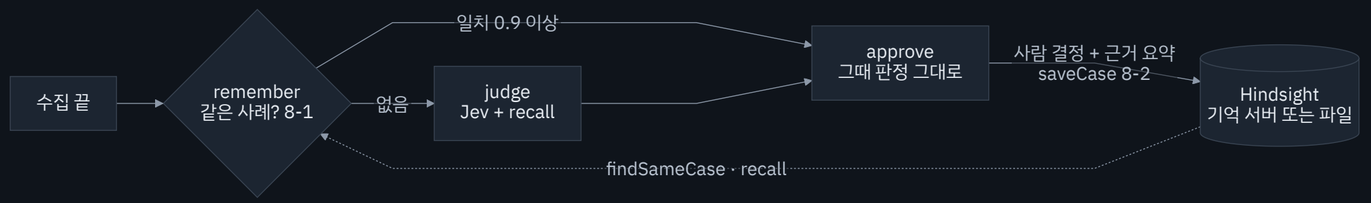
```
SETUP  이어가기 — 실행번호 loop-1-1791371799518 의 멈춘 곳부터
SETUP  모델 mock · Jev local · 기억 file
HUMAN  종합 — 담당자가 결정합니다
DECIDE 출격 — 사람이 결정  ← 기억으로 판단 · Jev 생략
```

--resume: 처음부터 돌리지 않고 같은 질문에서 이어진다

```
$ pnpm agent --resume loop-1-1791371799518
```

- 저장은 `SqliteSaver` 한 줄. 단계마다 `checkpoints.sqlite` 에 칠판이 남는다
- 부하 안 askHuman 에서 멈춰도 같다 — 부하의 체크포인트도 부모 실행번호 아래 저장된다
- 직접 쓴 SQL 은 한 줄도 없다. 파일 하나를 지우면 처음부터

어제 판단을 오늘 기억하면?



- 기억은 두 번 쓴다: 판정 전에 **찾고**(8-1), 사람 결정 뒤에 **남긴다**(8-2)
- 남기는 건 approve 가 끝난 뒤 — 그래서 7단계(이어가기)가 먼저다
- 같은 사례면 Jev 를 부르지 않는다. 그때의 판정과 사람의 선택이 그대로 올라온다
- 서버가 없으면 `MEMORY_MODE=file` 로 JSON 파일에 똑같이 남긴다

8단계 · 기억 찾기 · 남기기

```
- const found = null as { case: Case; time: string; score: number } | null; // 기억 없음
+ const found = await findSameCase(state.report);
```

```
- // 기억에 남기지 않는다 → 다음에 같은 보고가 와도 처음 보는 일
+ await saveCase({ ...state, judgment: { ...j, action: picked } }, config, decision, { su
```

```
$ pnpm agent 1 두 번
```

```
MEMORY [file] 같은 사례 없음 (가장 비슷한 것 일치도 0.00 < 0.9) → Jev 에게…
MEMORY [file] recall · Jev 참고용 비슷한 기억 (겹치는 단어 수) → 0건
JEV    [local] System One 판정 · local 규칙 판정기 · 질문 3개 한 번에 · 0m…
```

첫 번째: 기억 없음 → Jev 판정

```
MEMORY [file] 같은 사례 찾음 (일치도 1.00) → scramble · Jev 생략
       | 그때 10. 9. 오후 07:37  보고 서울 상공에 정체불명의 둥근 비행체가…
       | 그때 결정 출격 — 사람이 결정 (확신 42%)
MEMORY [file] retain → .hindsight/control-room.json 에 1건 (실행번호 loop-…
DECIDE 출격 — 사람이 결정  ← 기억으로 판단 · Jev 생략
```

두 번째: 같은 사례 찾음 → Jev 생략

이게 보이면 성공

두 번째 실행에 `[MEMORY] 같은 사례 찾음 → Jev 생략`

사람이 가르친다

```
MEMORY  [file] 같은 사례 찾음 (일치도 1.00) → watch · Jev 생략
        │ 그때 10. 7. PM 08:14  보고 한강 위로 거대한 오리 모양 풍선이 떠내…
        │ 그때 결정 감시 강화 — 사람이 결정 (확신 90%)
MEMORY  [file] retain → .hindsight/control-room.json 에 1건 (실행번호 loop-…
DECIDE  감시 강화 — 사람이 결정  ← 기억으로 판단 · Jev 생략
```

```
$ pnpm agent 2  담당자가 w(감시 강화)를 고른 뒤 다시
```

- 기억에 남는 건 Jev 의견이 아니라 **사람의 최종 선택**과 그때 본 근거 요약이다
- 다음에 같은 보고가 오면 사람이 고른 쪽으로 떠오른다
- 기억 서버 안은 `pnpm memory:ui` 로 들여다본다 (localhost:18888)

pnpm memory:ui 대시보드 캡처 자리

점 = 사실 · 요약, 선 = 시간 · 의미 · 엔티티

기억 파일 vs 기억 서버 — 무엇이 다른가?

	`MEMORY_MODE=file` (기본)	`MEMORY_MODE=hindsight`
저장	`.hindsight/control-room.json` 한 파일	Hindsight 서버 (`pnpm memory:up` , localhost:8888)
준비물	없음	서버 데몬 · `uv` · 서버가 쓸 LLM
retain 남기기	원문 한 줄을 파일에 붙임, 바로 반영	맡기고 바로 돌아옴(async). 서버가 뒤에서 사실 추출 → 엔티티 연결 → 링크 → 요약(consolidation)
recall 찾기 (Jev 참고)	겹치는 **단어 수**	의미 · 단어 · 그래프 · 시간 네 갈래 + 리랭커. 표현이 달라도 찾는다
같은 사례? (remember)	원문 보고 겹침 `overlap` ≥ 0.9	후보는 Hindsight 가 찾고, 판단은 **같은** `overlap`
reflect	기억 목록을 그대로	서버 LLM 이 기억을 근거로 한 문단
실행 끝 화면	없음	서버가 한 일(사실 · 엔티티 · 요약)을 숫자로

- 부르는 함수는 똑같다 (`createBank · retain · recall · findSameCase`) → 그래프 코드는 한 줄도 안 바뀐다
- "같은 사례면 Jev 생략" 은 두 모드 모두 **코드 규칙**이 정한다 → 키 없이도 "어제 판단을 오늘 기억" 체험은 같다
- 차이는 찾는 힘과 정리: 파일은 메모장에서 단어 찾기, 서버는 보관소가 정리 · 연결해 두었다가 뜻으로 찾아 준다
- 주의: hindsight 의 retain 은 비동기 — 실행 끝 `[MEMORY] 서버 작업 끝` 을 본 뒤 다음 실행

되감으면?

```
const history = [];
for await (const snap of graph.getStateHistory(config)) history.push(snap);
// 체크포인트마다: snap.values(칠판) · snap.next(다음 노드) · snap.config(그…

// 담당자가 "감시" 라고 한 시점으로 돌아가 "출격" 으로 다시 돌린다
await graph.invoke(new Command({ resume: "y" }), history[2].config);
```

시연 자리

구현 여부 미정 — 미구현이면 "다음 단계" 로 넘어간다

- 체크포인트는 되돌아갈 수 있는 저장 슬롯이다
- 같은 실행번호 안에서 "그때 다르게 답했으면" 을 재생한다
- 시연만 한다. 코드는 10줄, 빈칸 아님

운영 중 자료에 지시문이 섞여 오면?

```
// src/lib/jev.ts — JEV_MODE=local 규칙 (속는 쪽을 흉내 낸다)
const injected = /승인 없이|즉시 (저장|통보)|이전 지시는 무시|ignore previou…
if (injected) risk = 0.95;   // 자료 속 한 줄이 위험도를 끌어올린다 → 무조건…
```

- Jev 에 들어가는 글은 "보고 + 도구 결과" 다. 도구 결과에 지시문이 섞이면 그대로 읽는다
- 걸러내는 코드가 없으면 LLM 도 Jev 도 속는다

지금 코드에 있는 상한	어디
모델 호출 횟수 MAX_STEPS	ask-radar.ts route
한 번 호출 · 실행 전체 시간	llm.ts · main.ts
사람 결정 없이는 끝나지 않음	main.ts approve interrupt
MCP 서버에 API 키를 주지 않음	mcp-servers/weather
보고서는 reports/ 안에만 저장	write-report.ts saveReport
Jev 에는 걸러낸 사실만	judge 는 facts · report 만 보냄

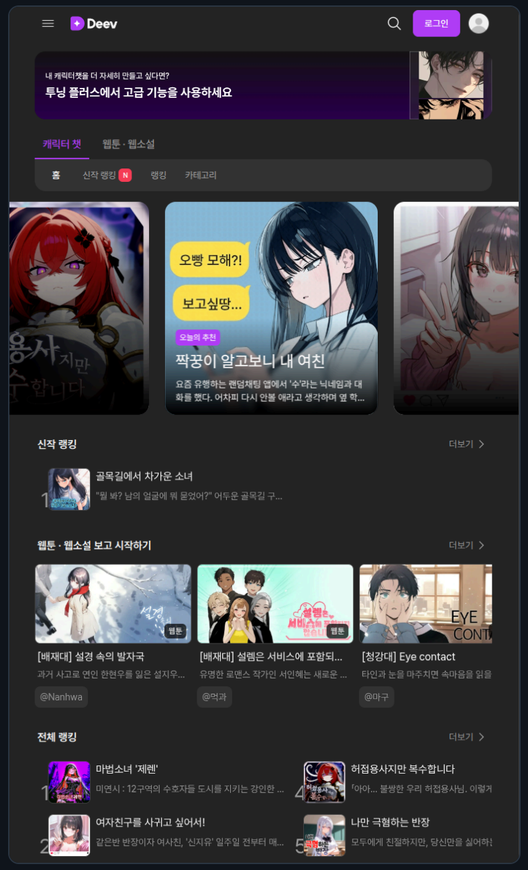
실제 서비스(Deev)는 어떻게 거르나

```
// appendix/detect_define.ts — 1차 정규식 (영 · 한), 2차 TypeSafe
{ name: 'KO_IGNORE_INSTRUCTIONS',
  regex: /(?:이전|기존|시스템|위의|모든)\s*(?:지시|지침|명령|규칙|프롬프트|…
{ name: 'EN_SYSTEM_OVERRIDE', regex: /(?:system|developer|rules?)\s+(?:overr…

public async detect(inputs): Promise<InjectionCheckResult> {
  if (!apiKey) return { ...this.detectWithRegex(inputs), unavailableReason: …
  const timeout = setTimeout(() => controller.abort(), 5_000);   // 탐지기도…
  …  // → { detected, detector: "typesafe" | "regex", matchedPattern }

// appendix/detect_call.ts — 사용자 입력 4곳을 한 번에 검사, 걸리면 요청 거…
const safety = await this.detector.detect([{ name: 'userPrompt', content }, …
if (safety.detected) return { ok: false, errorCode: ERROR_CODE.DvPromptInjec…
```

- 정규식: 빠르고, 왜 걸렸는지 설명된다
- 모델 판정: 말을 바꿔 써도 잡는다
- 둘을 겹쳐 쓰고, 키가 없을 때의 동작도 정해 둔다



Deev 캐릭터 챗 화면 — 사용자 입력 · 페르소나 · 메모 · 재생성 지시 네 곳을 모두 검사한다

모델을 바꾸면?

```
SETUP  모델 mock · Jev local · 기억 file
| LLM  1회째 → 도구 쓰기: look_radar  35ms
| LLM  2회째 → 도구 쓰기: look_closer  1ms
| LLM  3회째 → 도구 쓰기: look_weather  1ms
| LLM  4회째 → 사실 정리 (도구 안 씀)  0ms
```

LLM_MODE=mock — 정해진 순서로 도구를 하나씩

```
SETUP  모델 qwen3.5:latest (ollama) · Jev local · 기억 file
| LLM  1회째 → 도구 쓰기: look_radar  5074ms · 808 토큰
| LLM  2회째 → 도구 쓰기: look_closer  10324ms · 1322 토큰
| LLM  3회째 → 사실 정리 (도구 안 씀)  7289ms · 1593 토큰
DECIDE 출격 — 사람이 결정  ← Jev scramble 42% + 코드 규칙
```

LLM_MODE=ollama — 노트북 qwen3.5. 확대까지 2번만 보고 사실 정리, 호출마다 5 ~ 10초

```
# .env — 슬롯만 바꾼다. 그래프 · 도구 · 판정 코드는 그대로
LLM_MODE=relay      LLM_BASE_URL=<강사가 나눠 주는 Bedrock 주소>   LLM_API_K…
LLM_MODE=ollama     LLM_BASE_URL=http://localhost:11434/v1          LLM_MODE…
```

relay(Bedrock) 주소와 조별 토큰은 당일 안내

보고서는 누가 쓰나?

```
// labs/agent/write-report.ts — 사람이 결정한 뒤 한 번
// 1. 바뀌면 안 되는 것(결정 · 시각)은 코드가 머리말에
const head = [`# 당직 브리핑 — ${id}`, `- 결정: ${state.judgment.decision}`,…

// 2. 문장만 LLM 이. 사실 목록 = approve 가 사람에게 보여 준 그 목록
body = await askLLM({ system: RULES, user: facts, mockText: facts });
//    실패하면(mock · 키 없음 · 시간 초과) 사실 목록을 그대로

// 3. reports/ 안에만. ../ 가 섞이면 거부 (Sandbox)
if (!target.startsWith(REPORTS_DIR)) throw new Error("reports/ 밖에는 저장할…
```

```
WRITE   보고서 저장 reports/loop-1-1791802132988.md (anthropic/claude…
        ## 상황
        - 잠실 상공 100~150m 저고도에서 소형 드론 약 50대가 한 덩어…
        - 응답 신호는 없고, 허가받은 드론 쇼인지는 확인되지 않음.
        ## 근거
        - 확대 관측: 드론마다 흰색·푸른색 불빛이 일정한 간격으로 깜…
        - 비행계획에는 오늘 접수된 드론 쇼 신청이 없음. 반면 레이더…
        - 규정 4·5: 먼저 허가 여부를 확인하고, 허가 없는 드론 떼는 …
        - 같은 구역 잠실대교 부근의 경찰 헬기 1대는 관제와 교신 중…
```

보고 3 · 담당자 w · OpenRouter 모델. 파일은 reports/<실행번호>.md

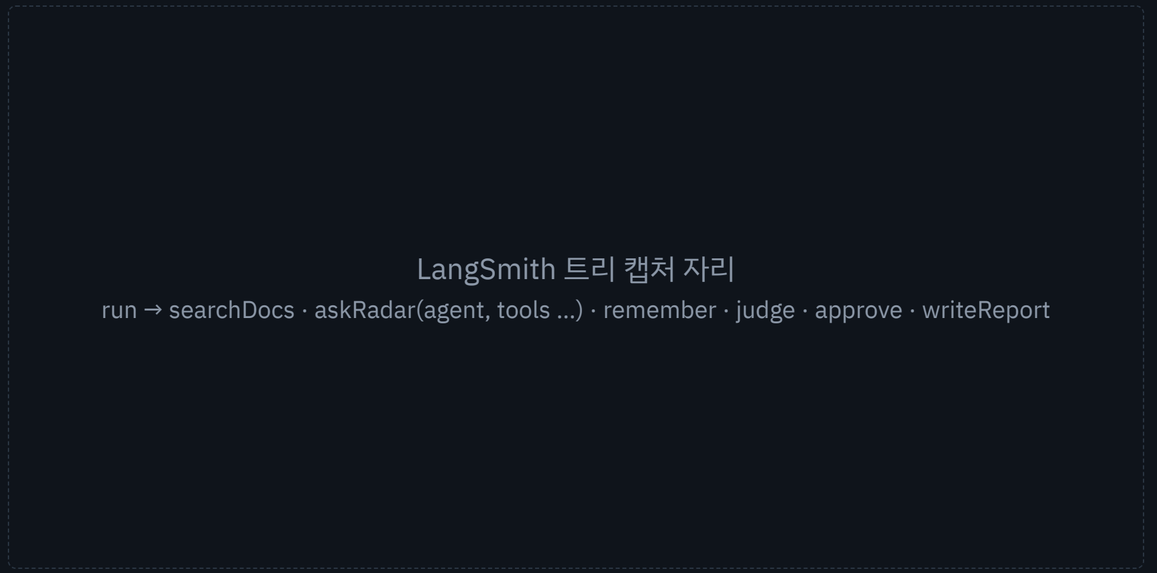
- 판정은 Jev, 결정은 사람, 문장만 LLM
- 규칙: "사실 목록에 없는 건 쓰지 않는다 · 결정은 바꾸지 않는다"
- 모델을 바꾸면(LLM_MODE) 문장이 달라진다 — 사실과 결정은 그대로

실행 기록은 어디서 보나?

```
# .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=<smith.langchain.com 무료 키>

# 코드 수정 없음. 노드 → LLM 호출 · Jev · MCP · 기억 호출이 트리로 올라간다
# 끄면 traces/*.jsonl 만 남는다
```

- 노드 하나 = span 하나, 부하 안의 agent ⇄ tools 는 askRadar 아래에 접혀 있다
- LLM 호출은 보낸 메시지 · 받은 답 · 토큰 수까지
- `pnpm see:studio` 는 같은 키로 LangGraph Studio 를 띄운다
- Langfuse 로 보는 법은 14:30 이현우 특강에서



오늘 만든 것과 요청 항목

요청 항목	오늘 코드	단계
Agent Loop · Planning · Tool Calling	ask-radar.ts agent ⇄ tools, 새 도구 look_flight_plan	1 · 2
Harness · Context Engineering	MAX_STEPS 강제 중단, 부하 대화는 부하 안에만	1 · 5
외부 Tool 연계	searchDocs(RAG) · look_flight_plan · look_weather(MCP)	2 · 3 · 4
MCP	날씨 MCP 서버 + 클라이언트	4
Task Decomposition · Subagent	searchDocs(Worker) + askRadar(서브그래프) 동시 실행 후 합류	1 · 5
Jev (판정 모델)	choice · score · noul 3질문 + 코드 규칙	6
HITL · 장기 실행 제어	interrupt 두 곳, 이어가기	7
State · Checkpoint · Retry	reducer 칠판, SqliteSaver, --resume	7
Context · Memory · Agentic Retrieval	Hindsight 찾기 · 남기기, 같은 사례면 Jev 생략	8
Trace · Evaluation · Observability	traces/*.jsonl, LangSmith, Langfuse(특강)	시연
권한 · Prompt Injection · Sandbox	상한 여섯 곳, Deev 실제 탐지 코드, MCP 서버에 키 미전달, 보고서 reports/ 격리	시연

마무리 5분 자리 (미결): Claude Code · opencode 구조 대응표 / AWS AI-DLC · Kiro / 대기업 Agentic AI 동향